

Natural Language Processing

02. Vector Semantics

Kenza BOUZIDI

`kenza.bouzidi.ext@u-pec.fr`

12 Décembre 2025

Motivation (1/2)

Comment les moteurs de recherche comme DuckDuckGo ou Google récupèrent-ils des documents pertinents à partir d'une query ?

Ces problèmes sont étudiés dans le domaine de :

- **l'Information Retrieval** : la science de la recherche d'information dans des collections de documents.

Comment une entreprise peut-elle traiter les réclamations que les utilisateurs laissent sur leurs portails web ?

Ces problèmes sont étudiés dans les domaines suivants :

- **Text Mining** : extraction automatique de connaissances à partir de texte.

Les deux sont étroitement liés au NLP ! (les frontières entre ces domaines ne sont pas nettes)

- La tokenisation est la tâche qui consiste à diviser une phrase ou un document en unités appelées *tokens*.
- Des transformations supplémentaires peuvent être appliquées, comme la suppression des caractères spéciaux (ponctuation), la conversion en minuscules, etc.[Manning et al., 2008].

Exemple

Entrée :

J'aime les chiens et les chats.

Tokens :

[j'aime] [les] [chiens] [et] [les] [chats]

- Un *type* est une catégorie de *token* correspondant à une seule séquence de caractères.
- On les obtient en identifiant les tokens uniques présents dans le document.

Exemple

Entrée :

J'aime les chiens et les chats.

Types :

[j'aime] [les] [chiens] [et] [chats]

Observation : Notez que le token *les* n'apparaît qu'une seule fois dans la liste des types, même s'il se répète dans la phrase.

Extraction du vocabulaire (1/2)

- Un *term* est un *type* normalisé.
- La normalisation est le processus consistant à créer des classes d'équivalence entre différents *types*. Cela sera clarifié dans les prochaines diapositives.
- Le vocabulaire V est l'ensemble des termes (tokens uniques normalisés) présents dans une collection de documents ou un corpus D .

Extraction du vocabulaire (2/2)

- Pour réduire la taille du vocabulaire et éliminer les termes qui n'apportent pas beaucoup d'information, on supprime les termes qui apparaissent avec une fréquence très élevée dans le corpus.
- Ces termes sont appelés *stopwords* et incluent les articles, pronoms, prépositions et conjonctions.
Exemple : [un, une, la, le, les, à, dans, sous, avec].¹

Exemple de suppression de stopwords

Entrée : Je n'aime pas la pizza

Output : [aime] [pizza]

¹Concepts liés : mots fonctionnels, mots de classe fermée.

Stemming (1/3)

Un processus de normalisation des termes dans lequel les termes sont transformés vers leur racine.

Règle	Exemple
SSES → SS	caresses → caress
IES → I	ponies → poni
SS → SS	caress → caress
S →	cats → cat

Table 1: Exemple de règles de stemming de l'Algorithme de Porter

Stemming (2/3)

Exemple

Entrée :

I like human languages and programming languages

Après stemming :

I like human languag and programm languag²

Le vocabulaire du document *d* après suppression des stopwords et application du stemming :

termId	value
t1	human
t2	languag
t3	program

²http://9ol.es/porter_js_demo.html

Avantages du *stemming* :

- Réduit la taille du vocabulaire
- Regroupe les mots liés morphologiquement
- Améliore le *recall* dans les systèmes de recherche d'information

Inconvénients :

- Peut produire des stems qui ne sont pas de vrais mots
- Peut regrouper des mots ayant des sens différents
- Ne tient pas compte du contexte du mot

- Une autre stratégie de normalisation des termes.
- Elle transforme également les mots en leurs formes racines.
- Elle réalise une analyse morphologique en utilisant des dictionnaires de référence (tables de correspondance) pour créer des classes d'équivalence entre les *types*.
- Par exemple, pour le token *studies*, une règle de stemming renverrait le terme *studi*, tandis qu'avec la lemmatisation on obtiendrait le terme *study*³.

³<https://blog.bitext.com/what-is-the-difference-between-stemming-and-lemmatization/>

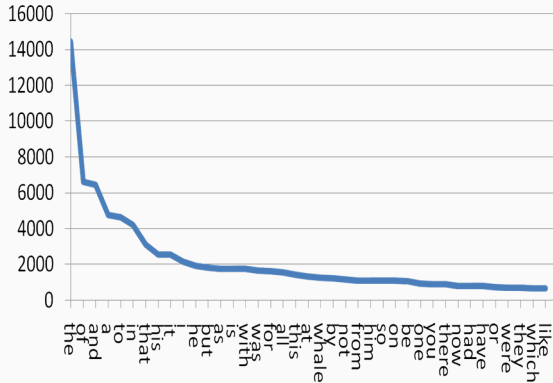
- Pour construire un vocabulaire efficace, nous devons comprendre comment les mots sont distribués dans les textes.
- La loi de Zipf, proposée par *George Kingsley Zipf* dans [Zipf, 1935], est une loi empirique qui décrit ce phénomène.
- Elle nous aide à :
 - Identifier les mots fréquents (potentielles *stopwords*)
 - Optimiser le stockage du vocabulaire
 - Comprendre des motifs linguistiques naturels

- Elle établit que la fréquence f d'un terme dans un corpus est inversement proportionnelle à son rang r dans une table de fréquences ordonnée :

$$f = \frac{cf}{r^\beta} \quad (1)$$

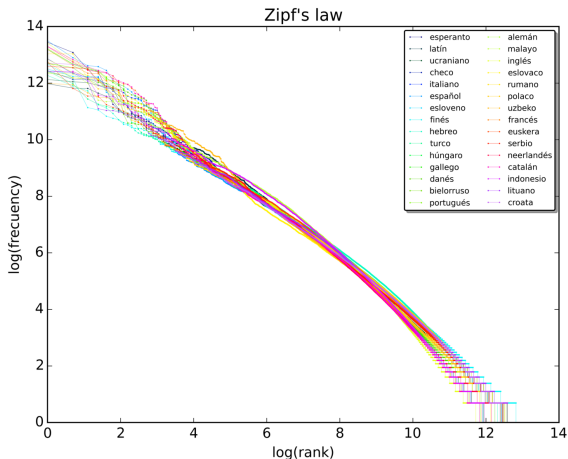
- Où :
 - cf est une constante dépendant de la collection
 - $\beta > 0$ est un facteur de décroissance
 - Si $\beta = 1$, la distribution suit exactement la loi de Zipf
- La loi est liée au principe du moindre effort : nous avons tendance à utiliser un ensemble réduit de mots très fréquents.

Loi de Zipf (3/4)



- Lister les mots les plus fréquents d'un corpus peut être utilisé pour construire une liste de *stopwords*.

Loi de Zipf (4/4)



Si l'on trace un graphique en échelle $\log\text{-}\log$, on obtient une ligne droite avec une pente de $-\beta$.

Représentation Vectorielle du Texte (1/2)

- Le traitement de texte nécessite de convertir les mots en nombres afin de pouvoir effectuer des calculs mathématiques.
- La représentation vectorielle permet :
 - De calculer des similarités entre documents
 - De regrouper des documents similaires
 - De classer des documents
 - De trouver des documents pertinents pour une requête

Représentation Vectorielle du Texte (2/2)

- Une approche de base consiste à *représenter* les documents comme des vecteurs de termes, où chaque terme correspond à une dimension vectorielle [Salton et al., 1975].
- Des documents contenant des mots différents et de longueurs différentes résideront dans le même espace vectoriel.
- Nous verrons ci-après une méthode classique pour représenter les mots sous forme de vecteurs, appelée *Bag of Words*, ainsi qu'une variante pondérée appelée *TF-IDF*.

Bag of Words (1/3)

- Le modèle de texte le plus simple et le plus courant est le modèle *Bag of Words* (BoW).
- Dans BoW, un document est représenté comme un sac (un multienemble) de ses mots.
- Caractéristiques principales :

Ce que BoW ignore	Ce que BoW prend en compte
L'ordre des mots La structure grammaticale La syntaxe	La fréquence de chaque mot La présence/absence des termes La distribution statistique

Bag of Words (2/3)

- La valeur de chaque dimension est un poids qui représente la pertinence du terme t_i dans le document d :

$$d_j \rightarrow \vec{d_j} = (w(t_1, d_j), \dots, w(t_{|V|}, d_j)) \quad (2)$$

Exemple

Texte original : Le chat mange du poisson et le chien mange de la viande

Représentation BoW :

Terme	Fréquence
le	2
chat	1
mange	2
poisson	1
et	1
chien	1
viande	1

Bag of Words (3/3)

Avantages	Inconvénients
Simple à implémenter Efficace computationnellement Fonctionne pour des tâches basiques Facile à mettre à l'échelle	Perd l'information contextuelle Ne capture pas la sémantique Gère mal les négations Forte dimensionnalité

- Malgré ses limites, Bag of Words est une représentation fondamentale qui sert de base à des modèles plus sophistiqués.
- Il est particulièrement utile dans des tâches comme :
 - Classification de documents
 - Recherche d'information
 - Analyse de sentiments basique

Term Frequency - Inverse Document Frequency (TF-IDF)

- TF-IDF est une mesure statistique qui évalue à quel point un mot est pertinent pour un document dans une collection.
- Elle combine deux métriques :
 - **Term Frequency (TF)** : Fréquence du mot dans le document
 - **Inverse Document Frequency (IDF)** : À quel point le mot est rare dans l'ensemble de la collection
- Intuition :
 - Si un mot apparaît souvent dans un document, il est probablement important (TF élevé)
 - Si un mot apparaît dans peu de documents, il est probablement plus distinctif (IDF élevé)

Term Frequency (TF)

- Term Frequency mesure la fréquence d'un terme dans un document.
- Il existe différentes manières de calculer le TF :

Variante	Formule
Fréquence brute	$tf_{i,j} = f_{i,j}$
Fréquence binaire	$tf_{i,j} = \begin{cases} 1 & \text{si } f_{i,j} > 0 \\ 0 & \text{sinon} \end{cases}$
Fréquence logarithmique	$tf_{i,j} = 1 + \log(f_{i,j})$
Fréquence normalisée	$tf_{i,j} = \frac{f_{i,j}}{\max_k f_{k,j}}$

Où $f_{i,j}$ est la fréquence du terme i dans le document j .

Inverse Document Frequency (IDF)

- L'IDF mesure à quel point un terme est commun ou rare dans l'ensemble de la collection.
- Il pénalise les termes qui apparaissent dans de nombreux documents.
- Formule de base :

$$idf_i = \log \left(\frac{N}{n_i} \right)$$

- N = nombre total de documents
- n_i = nombre de documents contenant le terme i

Terme	Docs contenant le terme	Total docs	IDF
le	98	100	0.02
algorithme	5	100	3.00
pelotillehue	1	100	4.61

- Les mots très courants comme “le” ont un IDF faible, tandis que les mots rares comme “pelotillehue” ont un IDF élevé.

Calcul Final du TF-IDF (1/2)

- Le poids final TF-IDF se calcule en multipliant $TF \times IDF$:

$$w_{i,j} = tf_{i,j} \times idf_i$$

Exemple

Document : "Le chat mange du poisson. Le chat dort"

Terme	TF	IDF	TF-IDF	Interprétation
le	2	0.02	0.04	Peu pertinent
chat	2	2.30	4.60	Très pertinent
mange	1	2.70	2.70	Pertinent
poisson	1	3.00	3.00	Pertinent
dort	1	2.70	2.70	Pertinent

- Les mots ayant un TF-IDF élevé sont les plus caractéristiques du document.
- Ce poids est utilisé pour construire les vecteurs de documents.
- Propriétés importantes du TF-IDF :
 - Il attribue un poids plus élevé aux termes fréquents dans peu de documents.
 - Il pénalise les termes qui apparaissent dans de nombreux documents.
 - Il est indépendant de la taille du document.
 - Il facilite la comparaison entre documents.

Similarité entre Vecteurs (1/2)

- Représenter des requêtes et des documents sous forme de vecteurs permet de calculer leur similarité.
- Une première approche serait d'utiliser la distance euclidienne.
- L'approche la plus courante consiste à calculer le cosinus de l'angle entre les deux vecteurs.
- Si les deux documents sont identiques :
 - L'angle serait 0
 - Leur cosinus serait 1
- Si les documents sont orthogonaux :
 - Le cosinus vaut 0

Similarité entre Vecteurs (2/2)

- La similarité cosinus se calcule de la manière suivante :

$$\cos(\vec{d}_1, \vec{d}_2) = \frac{\vec{d}_1 \cdot \vec{d}_2}{|\vec{d}_1| \times |\vec{d}_2|} = \frac{\sum_{i=1}^{|V|} (w(t_i, d_1) \times w(t_i, d_2))}{\sqrt{\sum_{i=1}^{|V|} w(t_i, d_1)^2} \times \sqrt{\sum_{i=1}^{|V|} w(t_i, d_2)^2}}$$

- On appelle souvent, à tort, cette mesure la *distance cosinus*. En réalité, c'est une mesure de similarité.
- Notez que la similarité cosinus normalise les vecteurs par leur norme euclidienne $\|\vec{d}\|_2$.

Similarité du Cosinus

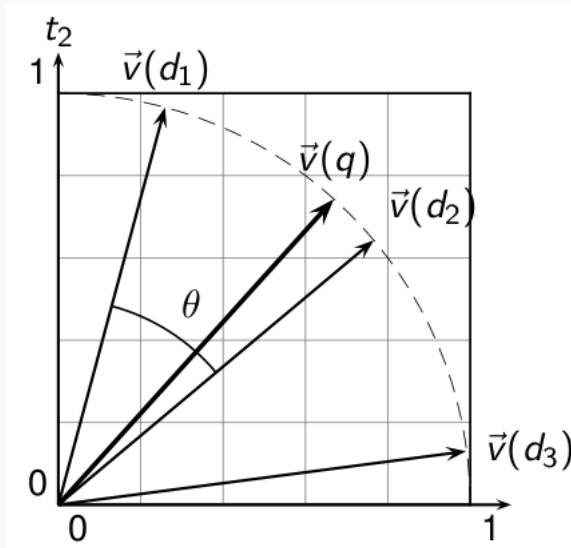


Figure 1: Similarité du cosinus.

Exercice

- Supposons que nous ayons 3 documents constitués à partir des séquences de termes suivantes :

$$d_1 \rightarrow t_4 t_3 t_1 t_4$$

$$d_2 \rightarrow t_5 t_4 t_2 t_3 t_5$$

$$d_3 \rightarrow t_2 t_1 t_4 t_4$$

- Construisez une matrice termes-documents de dimensions 5×3 en utilisant des poids *tf-idf* simples (sans normalisation).
- Nous vous recommandons d'abord de construire une liste indiquant le nombre de documents dans lesquels apparaît chaque terme (utile pour calculer les scores *idf*).
- Calculez ensuite les scores *idf* de chaque terme.
- Remplissez les cellules de la matrice avec les valeurs *tf-idf*.
- Quel est le document le plus proche de d_1 ?

Table 2: Matrice tf-idf

	d1	d2	d3
t1	0.176	0.000	0.176
t2	0.000	0.176	0.176
t3	0.176	0.176	0.000
t4	0.000	0.000	0.000
t5	0.000	0.954	0.000

N-grammes (1/2)

- Les n-grammes sont des séquences contiguës de n éléments (mots ou caractères).
- Types courants :
 - Unigrammes ($n=1$) : mots individuels
 - Bigrammes ($n=2$) : paires de mots consécutifs
 - Trigrammes ($n=3$) : triplets de mots consécutifs
- Applications dans BoW et TF-IDF :
 - Ils peuvent être utilisés comme termes à la place des mots individuels
 - Ils permettent de capturer des expressions ou des segments de phrases
 - Ils améliorent la précision dans les recherches de phrases exactes
- Inconvénients :
 - Plus grande dispersion des vecteurs
 - Coût computationnel plus élevé

Exemple

‘‘Le chat noir dort’’

- Unigrammes : [le] [chat] [noir] [dort]
- Bigrammes : [le chat] [chat noir] [noir dort]
- Trigrammes : [le chat noir] [chat noir dort]
- Dans BoW/TF-IDF avec n-grammes :
 - Chaque n-gramme constitue une dimension indépendante
 - On peut combiner différents niveaux de n
 - En pratique, on utilise généralement $n \leq 3$

- Les systèmes modernes de recherche d'information vont au-delà de la simple similarité vectorielle :
 - PageRank
 - Relevance Feedback
 - Query log mining
 - Google Knowledge Graph
 - Machine Learning
- La recherche d'information et le text mining sont moins préoccupés par la structure linguistique et davantage centrés sur la création d'algorithmes rapides et évolutifs [Eisenstein, 2018].

- Représenter les documents sous forme de vecteurs permet de calculer la similarité entre paires de documents.
- Les vecteurs Bag of Words et TF-IDF sont deux représentations qui :
 - Ne capturent pas la structure linguistique
 - Sont de haute dimension et très clairsemés
 - Peuvent être améliorés avec des n-grammes pour capturer des expressions multi-mots

- Eisenstein, J. (2018). *Natural Language Processing*. Technical Report, Georgia Tech.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press, New York, NY, USA.
- Salton, G., Wong, A., & Yang, C.-S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613–620.
- Zipf, G. K. (1935). *The Psychobiology of Language*. Houghton-Mifflin, New York, NY.